

LangChain vs LangGraph 实现 ReAct 的差异与 LangSmith 监控实践

五道口纳什 & Codex

2025-09-27

ReAct
create react agent
LangChain/LangGraph
langsmith

视频作者/频道: 五道口纳什

发布日期: 2025-09-27

视频时长: 17:09

视频链接: <https://www.bilibili.com/video/BV1n4n7znEDN>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 ReAct 模式回顾	2
1.1 什么是 ReAct	2
1.2 本视频定位	2
1.3 本章小结	2
2 LangChain 的 ReAct 实现: AgentExecutor	2
2.1 LangChain Agent 架构概览	2
2.2 AgentExecutor 的工作机制	3
2.2.1 关键特点	4
2.3 Agent Types	4
2.4 本章小结	4
3 LangGraph 的 ReAct 实现: 状态图引擎	4
3.1 LangGraph 设计哲学	4
3.2 用 LangGraph 实现 ReAct	5
3.3 LangGraph 的核心优势	6
3.3.1 显式状态管理	6
3.3.2 灵活的路由控制	6
3.3.3 细粒度控制	6
3.4 本章小结	7
4 LangSmith: Agent 行为监控与调试	7
4.1 LangSmith 是什么	7
4.2 核心功能	7
4.2.1 Tracing (追踪)	7
4.2.2 Debugging (调试)	8
4.2.3 Evaluation (评估)	8
4.2.4 Monitoring (监控)	8
4.3 LangSmith 实用小技巧	8
4.4 本章小结	8
5 实践: 两者的选择策略	8
5.1 什么时候用 LangChain AgentExecutor	8
5.2 什么时候用 LangGraph	9
5.3 从 LangChain 迁移到 LangGraph 的路径	9
5.4 本章小结	9
6 总结与延伸	9
6.1 核心回顾	9
6.2 向下一个系列承前启后	10
6.3 拓展阅读	10

1 ReAct 模式回顾

1.1 什么是 ReAct

ReAct (Reasoning + Acting) 是 LLM Agent 最基础的架构模式之一，由 Yao et al. 在 2022 年提出。其核心思想是让语言模型在**推理 (Reasoning)** 和**行动 (Acting)** 之间循环交替：

1. **Thought (思考)**：模型基于当前上下文分析下一步该做什么。
2. **Action (行动)**：调用一个工具 (Tool) 或执行一个具体操作。
3. **Observation (观察)**：获取工具返回的结果或环境反馈。
4. 回到步骤 1，直到问题解决。

ReAct 的核心价值

ReAct 将推理链与外部交互结合起来，让语言模型不再是”缸中之脑”，而是能够通过工具调用获取实时信息、执行计算、操作外部系统。

1.2 本视频定位

本视频是【Modern Agent】系列第二期，在上期讨论了 LLM Reasoning & Planning 的宏观分类之后，这一期回调到最经典朴素的 **ReAct**，对比两个最主流的框架实现：

- **LangChain**：原生的 AgentExecutor 实现
- **LangGraph**：基于状态图的灵活实现

同时引入 **LangSmith** 作为 Agent 行为监控和调试的工具。

视频描述原文

”ReAct 启发了现在的 Agent 设计与开发 (maybe 也启发了 OpenAI API 中的 function calling 功能的设计)，如同 CoT 启发了 prompt engineering 一样。”

1.3 本章小结

ReAct 是 Agent 架构的基石。理解它在不同框架中的实现差异，有助于我们在实际项目中做出正确的技术选型。

2 LangChain 的 ReAct 实现：AgentExecutor

2.1 LangChain Agent 架构概览

LangChain 是构建 LLM 应用最成熟的框架之一。其 Agent 系统的核心组件包括：

- **Agent**：决定下一步行动的 LLM 封装，包含 Prompt Template 和 Output Parser。
- **Tools**：Agent 可以调用的外部能力接口。
- **Toolkit**：相关工具的集合封装。

- **AgentExecutor**: 管理 Thought-Action-Observation 循环的执行器。
- **Memory**: 跨轮次的上下文存储。

2.2 AgentExecutor 的工作机制

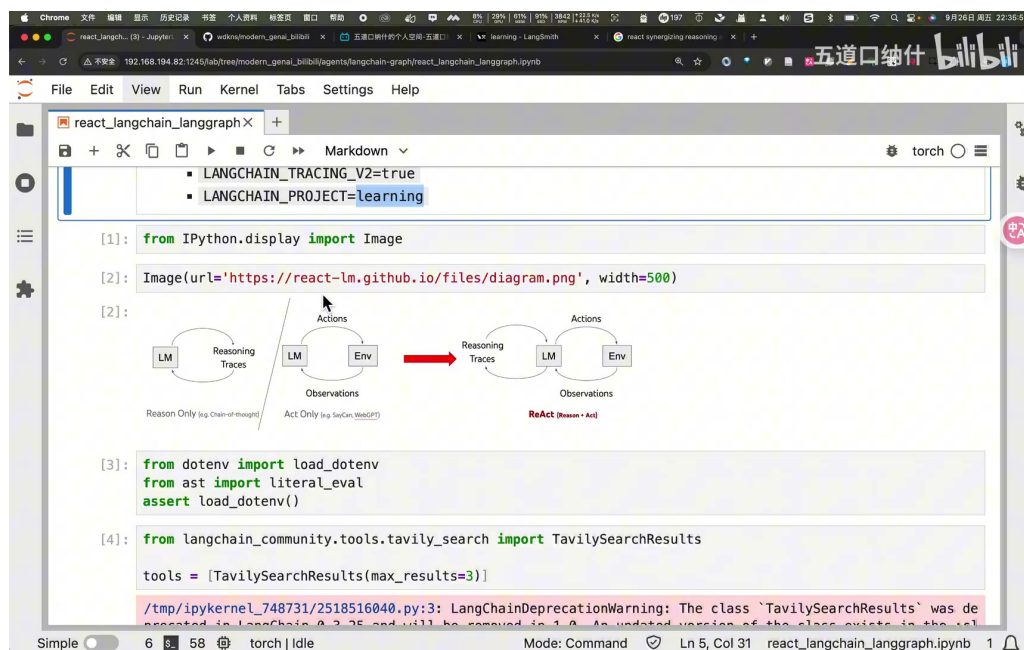


图 1: LangChain AgentExecutor 的 ReAct 循环架构¹

AgentExecutor 的核心是一个同步循环:

```

1  # LangChain 简化版 AgentExecutor 循环
2  while not done:
3      # 1. Agent 决定下一步
4      action = agent.plan(intermediate_steps)
5
6      if isinstance(action, AgentFinish):
7          return action.return_values
8
9      # 2. 执行工具调用
10     observation = tool(action.tool, action.tool_input)
11
12     # 3. 记录到中间步骤
13     intermediate_steps.append((action, observation))
14
15     # 4. 检查是否达到最大迭代次数
16     if len(intermediate_steps) >= max_iterations:
17         return handle_early_stopping()

```

Listing 1: LangChain AgentExecutor 核心逻辑

¹ 视频画面时间区间: 00:04:00-00:04:30

2.2.1 关键特点

- **线性循环**: Thought → Action → Observation → 循环, 直观易懂。
- **内置解析**: Agent 通过 Output Parser 将模型输出解析为结构化 Action。
- **最大迭代保护**: 通过 max_iterations 防止无限循环。
- **Early Stopping**: 支持提前终止策略。

2.3 Agent Types

LangChain 提供了多种预置 Agent 类型:

- **ZeroShotReactAgent**: 最基础的 ReAct 实现, 工具描述在 Prompt 中。
- **ConversationalReactAgent**: 带对话记忆的 ReAct。
- **StructuredChatAgent**: 支持多输入参数的复杂工具。
- **OpenAIFunctionsAgent**: 利用 OpenAI Function Calling 的原生能力。
- **XMLAgent**: 使用 XML 格式作为 Action 输出格式。

LangChain Agent 的局限性

- 循环逻辑固化在 AgentExecutor 中, 难以自定义中间步骤。
- 状态管理隐式——所有历史集中在 intermediate_steps 列表中。
- 分支逻辑复杂时需要大量 Hacks。
- 难以实现并行工具调用、条件路由等高级模式。

2.4 本章小结

LangChain 的 AgentExecutor 实现简洁直观, 适合快速原型开发和标准 ReAct 场景。但对于需要复杂控制流、并行执行或精细状态管理的场景, 其线性循环模型显得不够灵活。

3 LangGraph 的 ReAct 实现: 状态图引擎

3.1 LangGraph 设计哲学

LangGraph 是 LangChain 团队推出的新一代 Agent 框架, 核心思想是用图 (Graph) 来建模 Agent 的执行流程:

- **Nodes (节点)**: 代表一个具体的处理步骤 (Agent 决策、工具执行、条件判断)。
- **Edges (边)**: 定义节点之间的流转关系。
- **State (状态)**: 全局状态对象, 在节点之间传递和更新。

从 Executor 到 Graph

如果说 AgentExecutor 是一个硬编码的循环，LangGraph 则是让开发者自己绘制流程图。每个 Node 是一个独立逻辑单元，Edge 控制流转方向，State 在整个图中可见。

3.2 用 LangGraph 实现 ReAct

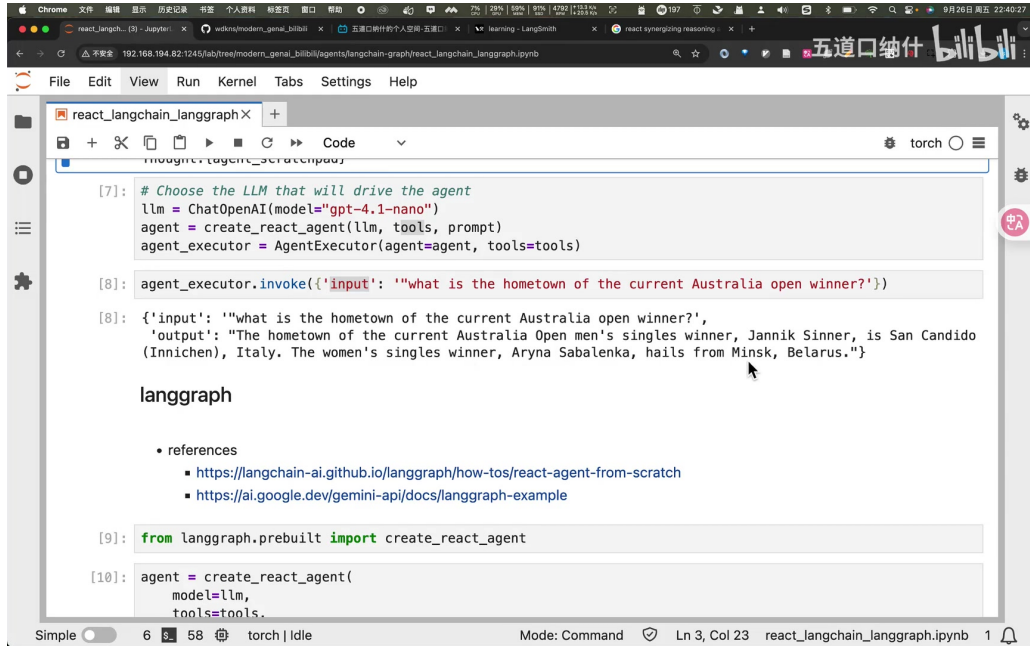


图 2: LangGraph 中 ReAct 的图结构: Agent Node → Tool Node → 条件路由²

```
1 from langgraph.graph import StateGraph, END
2 from typing import TypedDict, List
3
4 # 定义全局状态
5 class AgentState(TypedDict):
6     messages: List # 消息历史
7     next_action: str # 下一步动作类型
8
9 # 构建图
10 graph = StateGraph(AgentState)
11
12 # Agent 节点: LLM 决定下一步
13 graph.add_node("agent", call_agent)
14
15 # Tool 节点: 执行工具调用
16 graph.add_node("tool", run_tool)
17
18 # 条件边: 根据 agent 输出选择分支
19 graph.add_conditional_edges(
20     "agent",
```

²视频画面时间区间: 00:08:30–00:09:00

```

21     should_continue, # 路由函数
22     {"continue": "tool", "end": END}
23 )
24
25 # Tool 执行后返回 Agent
26 graph.add_edge("tool", "agent")
27
28 # 设置入口
29 graph.set_entry_point("agent")

```

Listing 2: LangGraph 实现 ReAct 的核心结构

3.3 LangGraph 的核心优势

3.3.1 显式状态管理

- State 类型化定义 (TypedDict / Pydantic)，类型安全。
- 所有节点共享 State，任意节点可以读写任意字段。
- 状态更新通过 Reducer 函数控制（如叠加消息、覆盖字段）。

3.3.2 灵活的路由控制

- **条件边**：根据 Agent 输出动态决定下一步（调用工具还是返回结果）。
- **并行边**：可以同时触发多个节点（并行工具调用）。
- **循环边**：天然支持循环 (Agent → Tool → Agent)，不需要额外循环计数逻辑。

3.3.3 细粒度控制

- 可以自定义任意中间节点（日志、检查、转换）。
- 支持中断 (Interrupt) 和断点 (Breakpoint) ——用于人机交互。
- 每个节点可以独立控制重试、错误处理、超时。

表 1: LangChain vs LangGraph ReAct 实现对比

维度	LangChain (AgentExecutor)	LangGraph (StateGraph)
循环模型	隐性 while 循环	显式图边
状态管理	intermediate_steps 列表	全局类型化 State
路由	硬编码（继续/停止）	条件边 + 自定义路由函数
并行调用	不支持	支持 (Parallel Edges)
人机交互	不支持原生	支持 Interrupt/Breakpoint
调试	日志级别控制	完整图 Trace
灵活性	中等（需 Hacks）	高（任意自定义 Node）
学习曲线	低	中等

3.4 本章小结

LangGraph 将 Agent 执行建模为有向图，提供了比 AgentExecutor 更灵活、更可控的架构。对于需要精细状态管理、复杂路由或人机交互的场景，LangGraph 是更合适的选择。

4 LangSmith: Agent 行为监控与调试

4.1 LangSmith 是什么

LangSmith 是 LangChain 生态中的**可观测性平台**（Observability Platform），专门用于 LLM 应用的调试、测试、评估和监控。

为什么需要 LangSmith

Agent 的行为具有**非确定性**和**长周期**的特点。一个问题可能经过多轮 Thought-Action-Observation 循环，传统的日志方式难以追踪。LangSmith 提供了可视化的 Trace 能力，让每个步骤都清晰可见。

4.2 核心功能

4.2.1 Tracing（追踪）

- 自动捕获每次 LLM 调用、工具执行、Agent 决策的完整链路。
- 以**树状结构**展示父子 Run 关系。
- 包含输入/输出、耗时、Token 用量等关键指标。

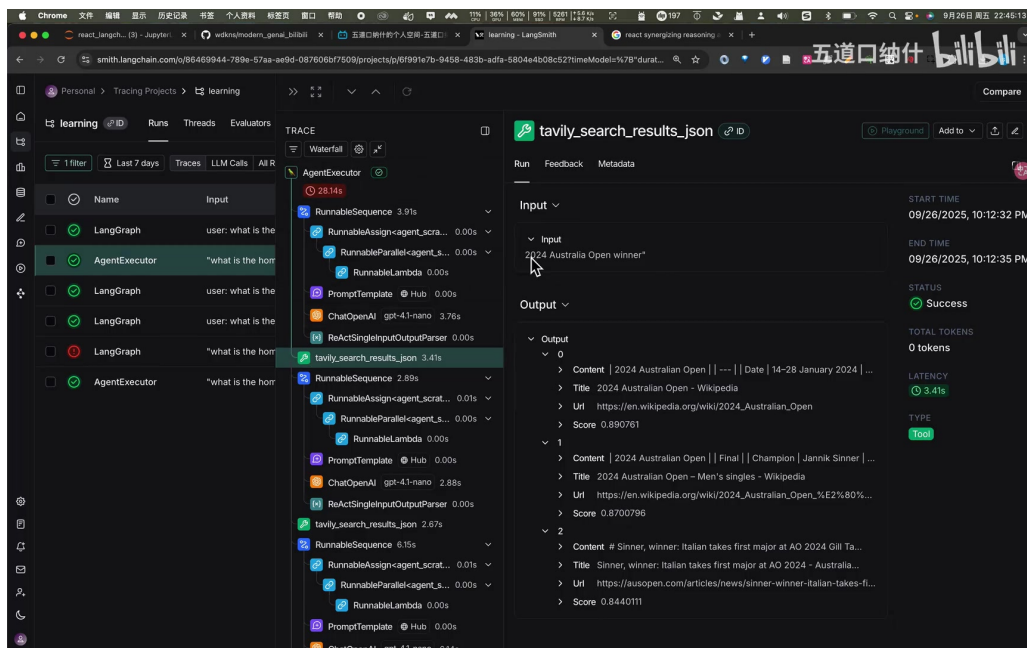


图 3: LangSmith Trace 界面：Agent 的每一步都清晰可追溯³

³ 视频画面时间区间：00:13:00-00:13:30

4.2.2 Debugging (调试)

- 查看每次 LLM 调用的完整 Prompt (含 System Prompt、History、User Input)。
- 检查 Tool Call 的输入和输出。
- 对比不同运行的参数和结果差异。

4.2.3 Evaluation (评估)

- 使用预定义或自定义指标评估 Agent 输出质量。
- 支持人工标注和自动化评估。
- 可以构建 Regression Test Suite。

4.2.4 Monitoring (监控)

- 实时仪表盘显示系统运行状态。
- 延迟、Token 消耗、错误率等指标监控。
- 可设置告警规则。

4.3 LangSmith 实用小技巧

视频中提到的 Tips

- **环境变量配置**: 设置 `LANGCHAIN_TRACING_V2=true` 和 `LANGCHAIN_API_KEY` 即可一键启用 Tracing。
- **项目隔离**: 不同项目使用不同的 `LANGCHAIN_PROJECT` 环境变量, 方便区分。
- **精打细算**: Token 消耗统计有助于优化 Prompt 长度和 Tool Call 策略。
- **异常排查**: 当 Agent 行为异常时, 先看 Trace 中的 LLM Raw Output 是否被正确解析。

实践建议

不要在生产环境才想起加 LangSmith——它应该在开发初期就集成。Trace 数据既是调试工具, 也是后期优化 Prompt 和 Agent 行为的重要依据。

4.4 本章小结

LangSmith 是 Agent 开发的”仪表盘”。结合 LangChain 或 LangGraph 使用, 可以在开发和运维全生命周期中保持对 Agent 行为的可见性。

5 实践：两者的选择策略

5.1 什么时候用 LangChain AgentExecutor

- **快速原型**: 几天内验证 Agent 概念的可行性。

- **标准 ReAct**: 需求不复杂, Thought-Action-Observation 线性循环即可满足。
- **团队初学**: 团队成员对 Agent 框架不熟悉, 需要低门槛上手。
- **单一工具**: 只调用少量简单工具。

5.2 什么时候用 LangGraph

- **复杂控制流**: 需要条件分支、并行执行、循环嵌套等。
- **人机交互**: 需要 Agent 在执行过程中暂停并等待人工确认。
- **精细状态**: 多个组件需要共享和更新复杂的状态。
- **生产级要求**: 需要完整的可观测性、错误恢复、重试机制。

核心选择标准

用 LangChain 还是 LangGraph, 不是”好与坏”的问题, 而是**复杂度匹配**的问题。如果你的 Agent 够简单, AgentExecutor 就够了; 如果需求在演进, LangGraph 提供了更长的保质期。

5.3 从 LangChain 迁移到 LangGraph 的路径

1. 先用 AgentExecutor 快速验证方案可行性。
2. 识别出需要自定义控制流的边界点。
3. 将 AgentExecutor 中的 Agent 和 Tools 直接复用到 LangGraph 的 Node 中。
4. 逐步替换硬编码逻辑为 Graph Edges 和 Condition Functions。

关键提醒

LangGraph 是 LangChain 团队开发的, 两者共享大量底层组件 (Tool 定义、LLM 封装、Prompt Template、Output Parser), 迁移成本远低于换用完全不同的框架。

5.4 本章小结

选型不是二选一的决断, 而是根据项目演进阶段动态调整的过程。

6 总结与延伸

6.1 核心回顾

1. **ReAct 是 Agent 的基础模式**: Thought $\rightarrow$ Action $\rightarrow$ Observation 循环, 启发了一整代 Agent 设计。
2. **LangChain AgentExecutor**: 简单直观的线性实现, 适合快速原型。
3. **LangGraph**: 基于状态图的灵活实现, 提供显式状态管理、条件路由、并行执行等高级能力。
4. **LangSmith**: 无处不在的可观测性工具, 是 Agent 开发不可或缺的调试和监控平台。

6.2 向下一个系列承前启后

从上期 (LLM Reasoning & Planning 综述) 到本期 (ReAct 框架实现差异), 这条学习路径清晰可见:

宏观分类 (第 1 期) → 具体实现 (第 2 期) → 进阶 **Workflow** (第 3 期: Plan-Execute-RePlan) → 结构化输出与工具调用 (第 4 期) → ...

值得思考的问题

- 为什么 ReAct 会在 2022 年之后成为 Agent 事实上的标准模式?
- LangGraph 的图模型是否有性能开销? 在什么场景下会成为瓶颈?
- 当 Agent 的行为越来越复杂时, 如何保证其可预测性和安全性?

6.3 拓展阅读

- **ReAct 原论文**: Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR 2023.
- **LangChain 官方文档**: <https://python.langchain.com/docs/modules/agents/>
- **LangGraph 官方文档**: <https://langchain-ai.github.io/langgraph/>
- **LangSmith 文档**: <https://docs.smith.langchain.com/>
- **Building Effective Agents**: Anthropic 发布的 Agent 设计最佳实践。